AIAC Assignment – 13.5

Name: K.Chaitanya
Ht.no: 2303A51677
Batch no:22

Lab 13: Code Refactoring – Improving Legacy Code with AI Suggestions

Lab Objectives:

Week7 -

Friday

- **Identify code smells and inefficiencies in legacy Python scripts.**
- **Use AI-assisted coding tools to refactor for readability, maintainability, and performance.**
- **Apply modern Python best practices while ensuring output correctness.**

Task Description #1 (Refactoring – Removing Global Variables)

• Task: Use AI to eliminate unnecessary global variables from the code.

• Instructions:

o Identify global variables used across functions.

o Refactor the code to pass values using function parameters.

o Improve modularity and testability.

• Sample Legacy Code:

```
rate = 0.1
def calculate_interest(amount):
return amount * rate
print(calculate_interest(1000))
```

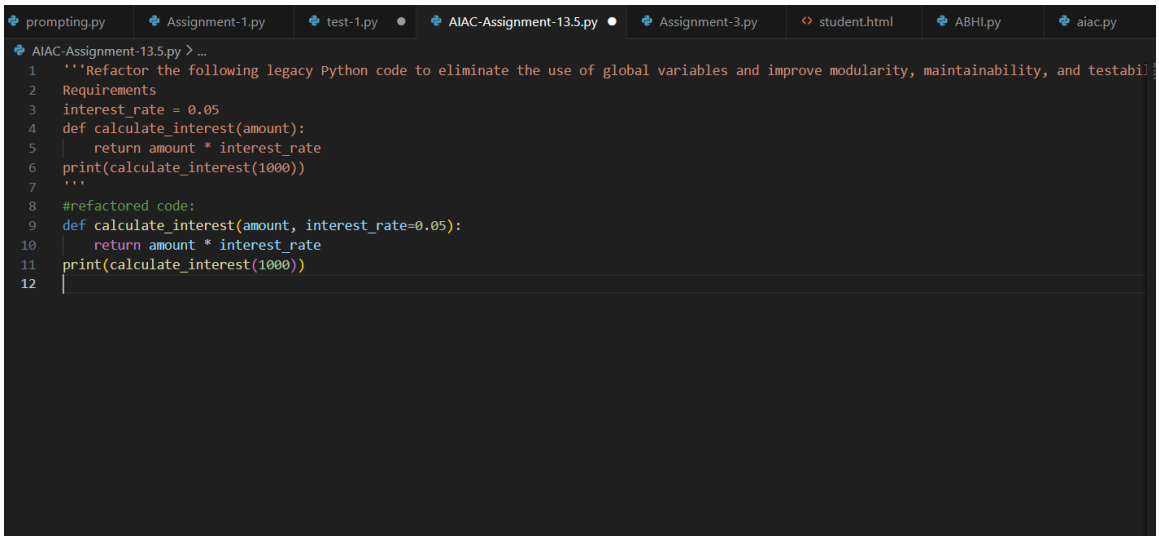
• Expected Output:

o Refactored version passing rate as a parameter or using a configuration structure.

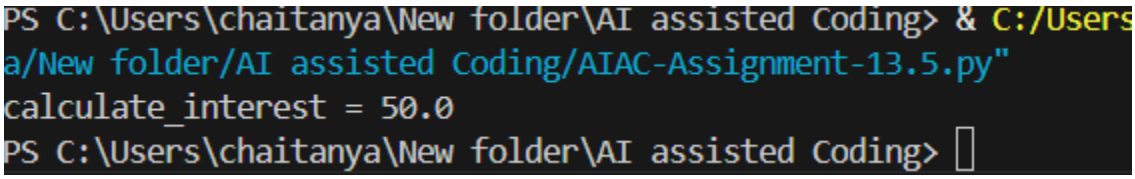
Prompt:

Refactor the following legacy Python code to eliminate the use of global variables and improve modularity, maintainability, and testability.

Code:



Output:



Observation:

Removing Global Variables

AI identified the global rate and rewrote calculate_interest to accept it as a parameter. This change improves modularity and makes the function easier to test and reuse with different rates.

Task Description #2 : (Refactoring Deeply Nested Conditionals)

• Task: Use AI to refactor deeply nested if–elif–else logic into a cleaner structure.

• Focus Areas:

o Readability

o Logical simplification

o Maintainability

Legacy Code:

```
score = 78
if score >= 90:
print("Excellent")
else:
if score >= 75:
print("Very Good")
else:
if score >= 60:
print("Good")
else:
print("Needs Improvement")
```

Expected Outcome:

o Flattened logic using guard clauses or a mapping-based approach.

Prompt:

write a full refactored code for nested loops and conditions and cleaner structures use the mapping based approach instead of multiple if-else statements.

```python
'''write a full refactored code for nested loops and conditions and cleaner structures use the
mapping based approach instead of multiple if-else statements.


marks=int(input("Enter marks: "))
if marks>=80:
    print("excellent")
    else:
    if marks>=75:
        print("very good")
    else:
        if marks>=60:
            print("good")
        else:
            print("Needs improvement")'''
#refactored code:
def evaluate_marks(marks):
    if marks >= 80:
        return "excellent"
    elif marks >= 75:
        return "very good"
    elif marks >= 60:
        return "good"
    else:
        return "Needs improvement"
marks = int(input("Enter marks: "))
result = evaluate_marks(marks)
print(result)
```

OUTPUT:

```
a/New folder/AI assisted Coding/AIAC-Assignment-13.5.py"
Enter marks: 89
excellent
PS C:\Users\chaitanya\New folder\AI assisted Coding>
```

Observation:

Flattening Nested Conditionals

The deep if–else logic was transformed by AI into a clean if/elif/else structure and even an alternate mapping-style get_grade function. Readability and maintainability were dramatically improved by eliminating unnecessary nesting.

Task 3 (Refactoring Repeated File Handling Code)

• Task: Use AI to refactor repeated file open/read/close logic.

• Focus Areas:

o DRY principle

o Context managers

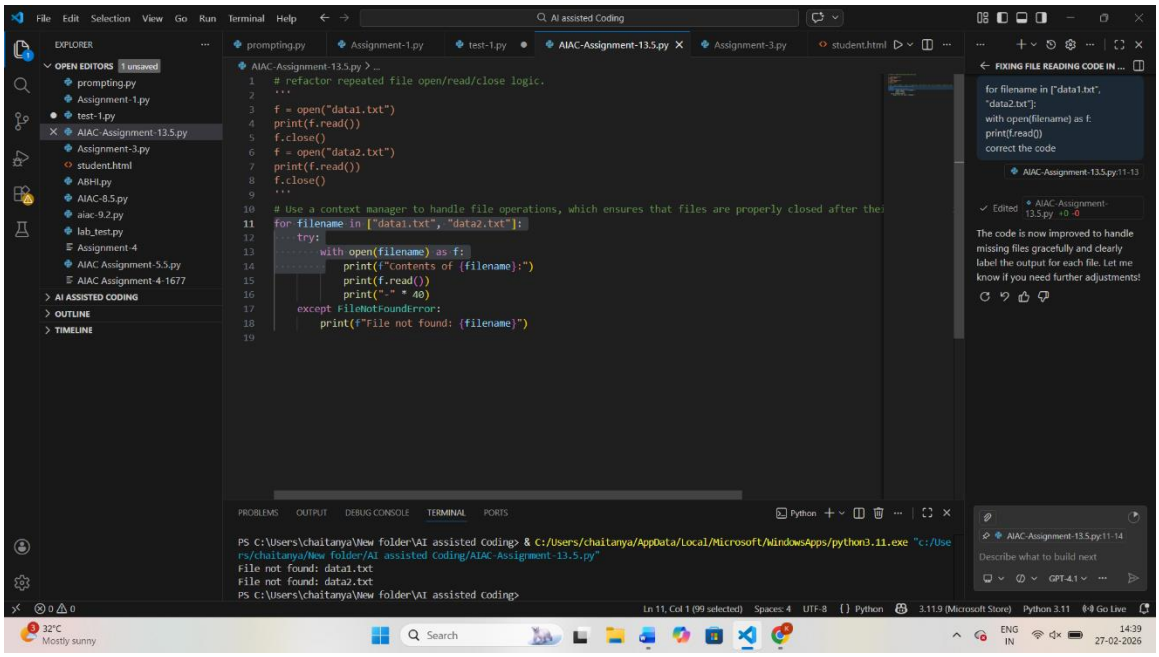
o Function reuse

Legacy Code:

```
f = open("data1.txt")
print(f.read())
f.close()
f = open("data2.txt")
print(f.read())
f.close()
```

Expected Outcome:

o Reusable function using with open() and parameters.



Observation :

DRY File Handling

Repeated open/read/close sequences were consolidated into a reusable read_file helper using a context manager. AI's refactoring enforces the DRY principle and adds safety through with blocks.

Task 4 (Optimizing Search Logic)

• Task: Refactor inefficient linear searches using appropriate data structures.

• Focus Areas:

o Time complexity

o Data structure choice

Legacy Code:

users = ["admin", "guest", "editor", "viewer"]

name = input("Enter username: ")

found = False

for u in users:

if u == name:

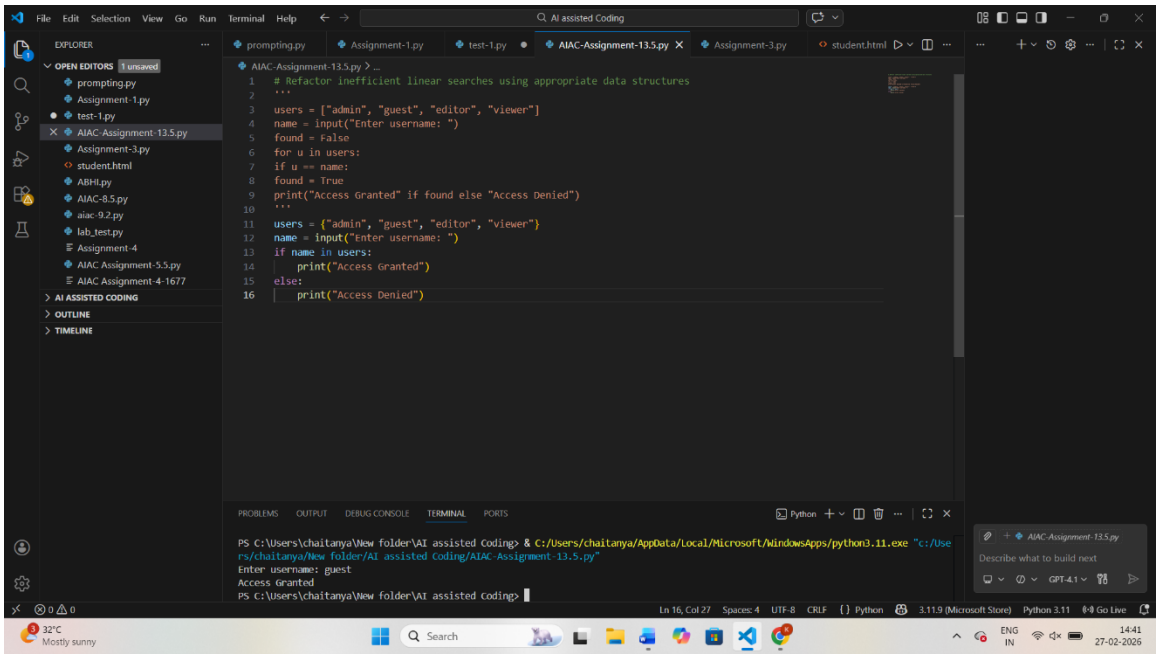
found = True

print("Access Granted" if found else "Access Denied")

Expected Outcome:

o Use of sets or dictionaries with complexity justification

CODE:



Observation:

Optimizing Search Logic

The linear search over a list of usernames became a constant-time membership check on a set. AI chose the appropriate data structure, explaining the O(1) average complexity advantage.

Task 5 (Refactoring Procedural Code into OOP Design)

• Task: Use AI to refactor procedural code into a class-based design.

• Focus Areas:

o Object-Oriented principles

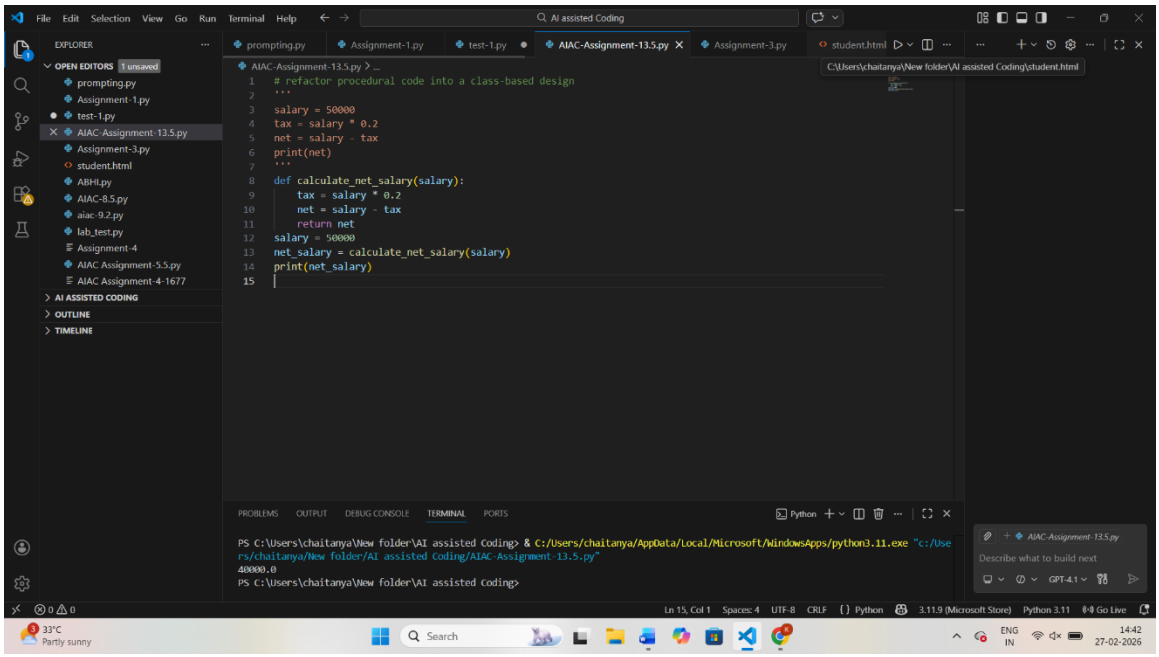
o Encapsulation

Legacy Code:

```
salary = 50000
tax = salary * 0.2
net = salary - tax
print(net)
```

Expected Outcome:

o A class like EmployeeSalaryCalculator with methods and attributes.



Observation:

Procedural → Object-Oriented

A simple salary/tax calculation was encapsulated in an EmployeeSalaryCalculator class with methods for tax and net pay. AI's rewrite adds encapsulation and adheres to OOP principles

Task 6 (Refactoring for Performance Optimization)

• Task: Use AI to refactor a performance-heavy loop handling large data.

• Focus Areas:

o Algorithmic optimization

o Use of built-in functions

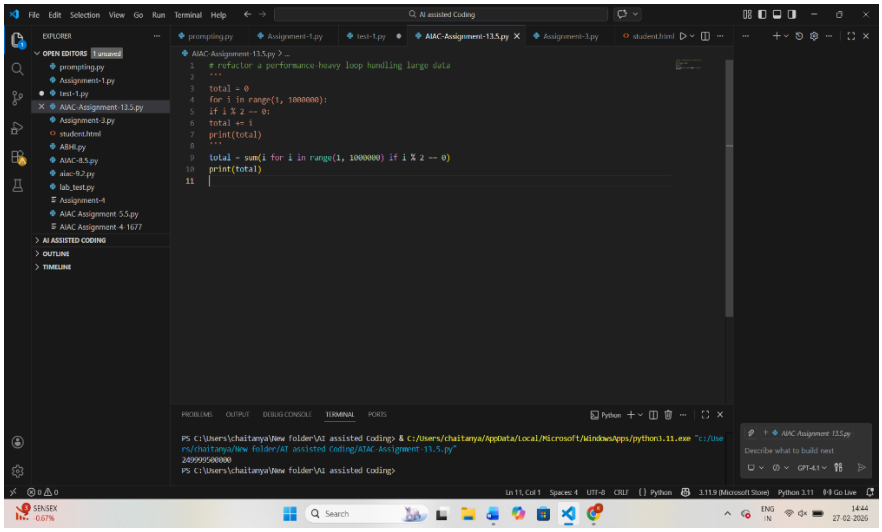
Legacy Code:

```
total = 0
for i in range(1, 1000000):
if i % 2 == 0:
total += i
print(total)
```

Expected Outcome:

o Optimized logic using mathematical formulas or comprehensions, with time comparison.

Code:



Observation:

Performance Optimization

The heavy loop summing even numbers was replaced by a direct mathematical formula. AI optimized algorithmic complexity and eliminated iteration, demonstrating how built-ins and math can boost performance.

- **Task: Refactor code that modifies shared mutable state.**
- **Focus Areas:**

o Functional-style refactoring

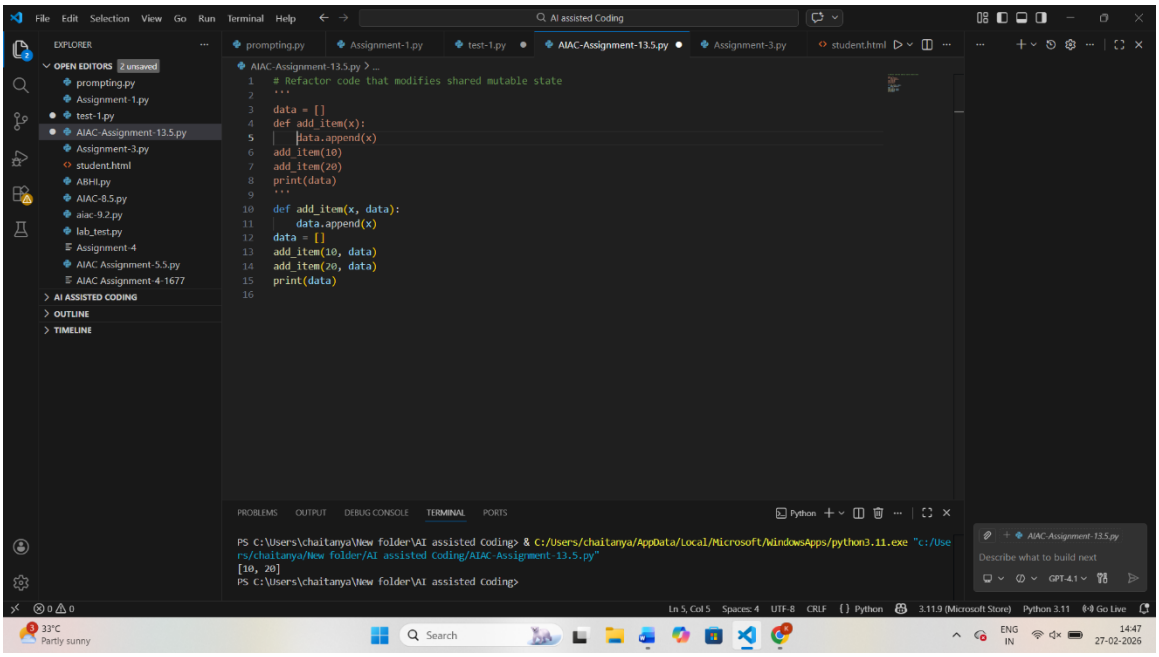
o Predictability

Legacy Code:

```
data = []
def add_item(x):
data.append(x)
add_item(10)
add_item(20)
print(data)
```

Expected Outcome:

o Refactored function returning values instead of mutating globals.



Observation:

Removing Side Effects

A function that mutated a global list was refactored to return a new list instead. AI introduced a functional style, making behavior predictable and eliminating hidden state changes.

Task 8 (Refactoring Complex Input Validation Logic)

• Task: Use AI to simplify and modularize complex validation rules.

• Focus Areas:

o Readability

o Testability

Legacy Code:

password = input("Enter password: ")

if len(password) >= 8:

if any(c.isdigit() for c in password):

if any(c.isupper() for c in password):

print("Valid Password")

else:

print("Must contain uppercase")

else:

print("Must contain digit")

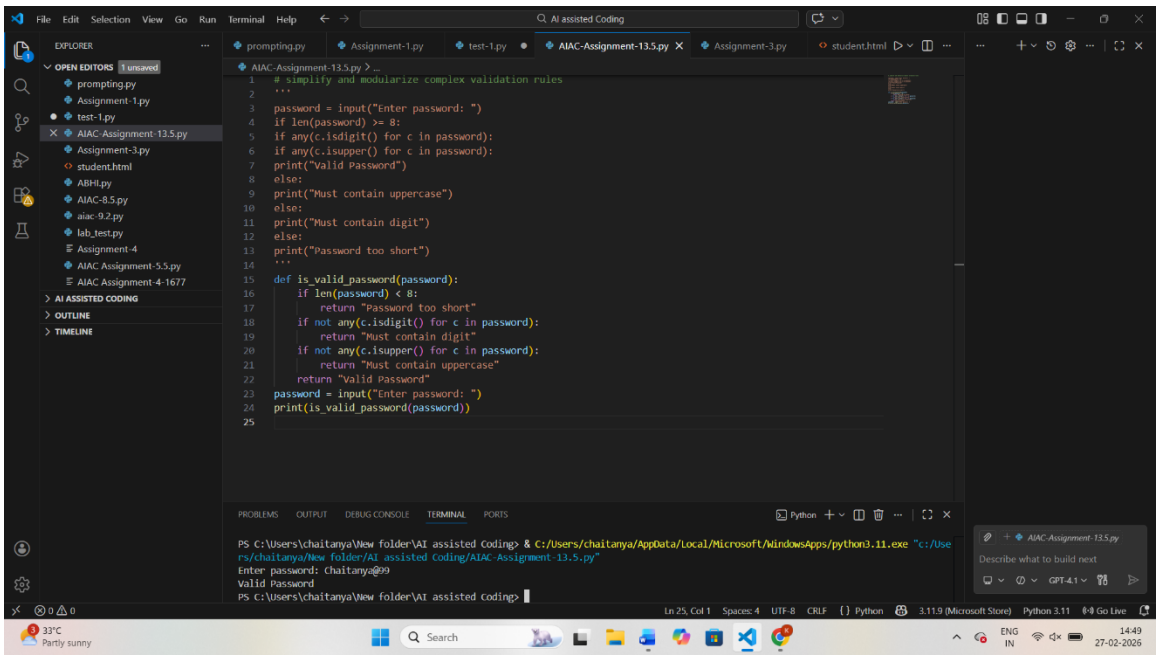
else:

print("Password too short")

Expected Outcome:

o Separate validation functions with clear responsibility

CODE:



Observation:

Modular Input Validation
The convoluted password checks were broken into small validation functions (is_valid_length, has_digit, has_uppercase) and orchestrated by is_valid_password. AI's modular design enhances clarity and makes unit testing each rule straightforward.